MODULE 1 — BASES DE DONNÉES AVEC R

DBI, dbplyr et requêtes performantes

OBJECTIF

Connexion → Requête SQL → KPI fiable

POURQUOI INTERROGER LA BASE DIRECTEMENT ?

Éviter les extractions lourdes et fragiles

- Les données volumineuses restent côté serveur
- Le filtrage se fait avant transfert
- Moins de mémoire locale consommée
- Meilleure traçabilité des requêtes

CONNEXION DBI

Pattern standard, quel que soit le SGBD

```r
library(DBI)
library(RSQLite)

con <- dbConnect(RSQLite::SQLite(), "business.db")
dbListTables(con)

dbDisconnect(con)
```

OBJETS DISTANTS AVEC DBPLYR

Code dplyr, exécution SQL

```r
library(dplyr)
library(dbplyr)

ventes_tbl <- tbl(con, "ventes")

kpi <- ventes_tbl %>%
  filter(date >= "2025-01-01") %>%
  group_by(region) %>%
  summarise(ca_total = sum(prix_net, na.rm = TRUE), .groups = "drop")

show_query(kpi)
kpi %>% collect()
```

LAZY EVALUATION

La requête part au serveur au bon moment

- `tbl()` crée une référence, pas une copie des données
- Les verbes dplyr construisent la requête SQL
- `collect()` déclenche l'exécution et rapatrie le résultat
- Bonne pratique : collecter uniquement la table déjà agrégée

BONNES PRATIQUES DE PERFORMANCE

- Filtrer le plus tôt possible dans le pipeline
- Limiter les colonnes avec `select()`
- Éviter les jointures inutiles
- Contrôler le SQL généré avec `show_query()`
- Fermer proprement la connexion après traitement

JOINTURE DISTANTE (APERÇU)

```r
clients_tbl <- tbl(con, "clients")

analyse_tbl <- ventes_tbl |>
  left_join(clients_tbl, by = "client_id") |>
  group_by(region, segment) |>
  summarise(ca = sum(prix_net, na.rm = TRUE), .groups = "drop")
```

POINTS DE VIGILANCE

- Types incohérents entre tables (texte vs numérique)
- Colonnes dates mal parsées
- Clés non uniques causant des duplications silencieuses
- Droits SQL insuffisants pour certaines tables

ATELIER DU MODULE

- Connecter une base de ventes
- Calculer le CA mensuel par région
- Produire le Top 10 clients de la période
- Exporter le résultat final en CSV

RÉCAPITULATIF

- ✓ DBI pour la connexion multi-SGBD
- ✓ dbplyr pour écrire en dplyr et exécuter en SQL
- ✓ collect() uniquement en fin de pipeline
- ✓ Focus performance et fiabilité des extractions